

Vincoli interrelazionali

Possono essere definiti attraverso i costrutti sintattici:

FOREIGN KEY e REFERENCES Permettono di definire vincoli di integrità referenziale

CHECK Lo vedremo (Vincoli complessi)

Si hanno due sintassi:

- per singoli attributi
- per più attributi

E' possibile definire politiche di reazione alle violazioni.

Vincolo di integrità referenziale

- Un vincolo di **integrità referenziale** (“**foreign key**”) fra gli attributi X di una relazione R_1 e un’altra relazione R_2 impone ai valori su X in R_1 di comparire **come valori della chiave primaria di R_2**

- Un vincolo di **integrità referenziale** (“**foreign key**”) fra gli attributi $X = \{A1, A2, \dots\}$ di una Tabella Figlio (Interna) e un’altra Tabella Padre (Esterna) impone ai valori degli attributi X nella tabella Figlio di comparire **come valori della chiave primaria della Tabella Padre**

Vincoli interrelazionali : Integrità referenziale

Esprime un legame gerarchico (padre / figlio) fra tabelle.

Alcuni attributi della tabella **figlio** sono definiti **FOREIGN KEY** e si devono riferire (**REFERENCES**) ad alcuni attributi della tabella **padre** che costituiscono una chiave (devono essere UNIQUE e NOT NULL oppure PRIMARY KEY).

I valori contenuti nella **FOREIGN KEY** devono essere **sempre** presenti nella tabella padre.

La tabella Padre è anche detta tabella esterna

La tabella Figlio è anche detta tabella interna

Integrità referenziale Esempio

Infrazioni

Tabella Figlio

<u>Codice</u>	Data	Vigile	Prov	Numero
34321	1/2/95	3987	MI	39548K
53524	4/3/95	3295	TO	E39548
64521	5/4/96	3295	PR	839548
73321	5/2/98	9345	PR	839548



Vigili

Tabella Padre

<u>Matricola</u>	Cognome	Nome
3987	Rossi	Luca
3295	Neri	Piero
9345	Neri	Mario

Integrità referenziale Esempio

Infrazioni (Figlio)

<u>Codice</u>	Data	Vigile	<u>Prov</u>	<u>Numero</u>
34321	1/2/95	3987	MI	39548K
53524	4/3/95	3295	TO	E39548
64521	5/4/96	3295	PR	839548
73321	5/2/98	9345	PR	839548

Auto
(Padre)

<u>Prov</u>	<u>Numero</u>	Cognome	Nome
MI	39548K	Rossi	Mario
TO	E39548	Rossi	Mario
PR	839548	Neri	Luca

Integrità referenziale: Sintassi

Due sintassi:

1. nella parte di definizione degli attributi con il costrutto sintattico **REFERENCES**:

AttrFiglio **CHAR(3)** **REFERENCES** *TabellaPadre*(*AttrPadre*)

2. oppure dopo le definizioni degli attributi con i costrutti **FOREIGN KEY** e **REFERENCES**:

FOREIGN KEY (*AttrFiglio*)
REFERENCES *TabellaPadre*(*AttrPadre*)

Vincoli interrelazionali : Integrità referenziale

Quando si hanno più attributi da riferire, si utilizza sempre FOREIGN KEY:

```
FOREIGN KEY (AttrFiglio1 {,AttrFiglio2} )  
    REFERENCES TabellaPadre(AttrPadre1 {,AttrPadre2})
```

Se si omettono gli attributi destinazione, vengono assunti quelli della chiave primaria.

```
AttrFiglio CHAR(3) REFERENCES TabellaPadre
```

Integrità referenziale: esempio di sintassi

```
CREATE TABLE Infrazioni(  
  Codice    CHAR(6)    NOT NULL PRIMARY KEY,  
  Data      DATE       NOT NULL,  
  Vigile    INTEGER    NOT NULL REFERENCES Vigili(Matricola),  
  Provincia CHAR(2),  
  Numero    CHAR(6) ,  
  FOREIGN KEY(Provincia, Numero)  
    REFERENCES Auto(Provincia, Numero)  
)
```


Il problema dell'aggiornamento

<u>Matr</u>	Nome	Città	CDip
34321	Luca	Mi	Inf
53524	Giovanni	To	Mat
64521	Emilio	Ge	Ing
73321	Francesca	Vr	Mat

Studente

Esame

<u>Matr</u>	<u>CodCorso</u>	Data	Voto
34321	1	25/02/01	28
34321	2	14/12/01	30
64521	1	12/03/02	24
73321	4	18/01/02	18

Se eliminiamo da Studente la tupla con matricola 34321, cosa succede alla tabella Esame?

Il problema delle violazioni

- Se viene eseguita una operazione di **aggiornamento** che porta alla **violazione di un vincolo di integrità referenziale**, (per ogni vincolo visto) la base di dati diventa non valida.
- Sono perciò definite un insieme di politiche per evitare che questo accada.
- Per i vincoli di integrità referenziale SQL permette di scegliere delle reazioni da adottare in caso di violazioni.
- Tali politiche vanno **dichiarate nella dichiarazione DDL dello schema**
- **Per tutti gli altri vincoli** visti fino ad ora, a seguito di una violazione, il comando di aggiornamento viene **rifiutato** segnalando l'errore.

Il problema delle violazioni

Si possono introdurre violazioni operando sulle righe della tabella padre (esterna) o sulle righe della tabella figlio (interna).

Tipo di violazione

Inserimento o modifica di tupla che viola il vincolo sulla **tabella figlio** (interna)



Politica di reazione

In entrambi i casi il comando non viene eseguito

Inserimento o modifica di tupla che viola il vincolo sulla **tabella padre** (esterna)



Quattro politiche possibili (vedi seguito)

Motivo della asimmetria: la tabella padre è la rilevante per l'integrità complessiva della base di dati

Reazione a violazioni sulla tabella padre

CASCADE: L'operazione di modifica/ cancellazione viene propagata alla tabella interna

SET NULL: NULL nella tabella figlio in entrambi i casi

SET DEFAULT: default nella tabella figlio in entrambi i casi

NO ACTION: rifiutata in entrambi i casi

Reazione a violazioni sulla tabella padre

Le politiche di reazione possono essere definite in modo diverso per eventi di modifica/cancellazione

Esempio:

```
Dipart CHARACTER (15) REFERENCES Dipartimento (NomeDip)  
ON DELETE SET NULL  
ON UPDATE CASCADE
```

Il problema dell'aggiornamento

<u>Matr</u>	Nome	Città	CDip
34321	Luca	Mi	Inf
53524	Giovanni	To	Mat
64521	Emilio	Ge	Ing
73321	Francesca	Vr	Mat

Studente

Esame

<u>Matr</u>	<u>CodCorso</u>	Data	Voto
34321	1	25/02/01	28
34321	2	14/12/01	30
64521	1	12/03/02	24
73321	4	18/01/02	18

Se eliminiamo da Studente la tupla con matricola 34321, cosa succede alla tabella Esame?

Vincoli interrelazionali : Cancellazioni

Cosa succede degli esami se si cancella uno studente?

CASCADE: si cancellano anche gli esami dello studente

SET NULL: si pone a null la matricola dei relativi esami

SET DEFAULT: si pone a valore di default la matricola dei relativi esami

NO ACTION: si impedisce la cancellazione dello studente

Vincoli interrelazionali : UPDATE

Cosa succede degli esami se si modifica la matricola di uno studente?

CASCADE: si modifica la matricola degli esami dello studente

SET NULL: si pone a null la matricola dei relativi esami

SET DEFAULT: si pone al valore di default la matricola dei relativi esami

NO ACTION: si impedisce la modifica della matricola dello studente

Vincoli interrelazionali : Esempio

```
CREATE TABLE Esame (  
  Matr      CHAR(6),  
  CodCorso  CHAR(6),  
  Data      DATE      NOT NULL,  
  Voto      Voto,  
  PRIMARY KEY (Matr, CodCorso)  
  FOREIGN KEY (Matr) REFERENCES Studente  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
)
```

Vincoli interrelazionali : Esempio

```
CREATE TABLE Esame (  
  Matr      CHAR(6),  
  CodCorso  CHAR(6),  
  Data      DATE      NOT NULL,  
  Voto      Voto,  
  PRIMARY KEY (Matr, CodCorso)  
  FOREIGN KEY (Matr) REFERENCES Studente  
    ON DELETE CASCADE  
    ON UPDATE CASCADE  
  FOREIGN KEY (CodCorso) REFERENCES Corso  
    ON DELETE NO ACTION  
    ON UPDATE CASCADE  
)
```

Vincoli Interrelazionali

Definiamo tre tabelle con le informazioni degli esami sostenuti dagli studenti:

- Tabella Studente
- Tabella Esame
- Tabella Corso

Definizione Tabella Studente

```
CREATE TABLE Studente (  
  Matr      CHAR(6)          PRIMARY KEY,  
  Nome      VARCHAR(30)     NOT NULL,  
  Città     VARCHAR(20),  
  CDip      CHAR(3)  
)
```

<u>Matr</u>	Nome	Città	CDip
34321	Luca	Mi	Inf
53524	Giovanni	To	Mat
64521	Emilio	Ge	Ing
73321	Francesca	Vr	Mat

Definizione Tabella Esame

```
CREATE TABLE Esame (  
  Matr          CHAR(6),  
  CodCorso      CHAR(6),  
  Data          DATE          NOT NULL,  
  Voto          Voto,  
  PRIMARY KEY (Matr, CodCorso)  
)
```

<u>Matr</u>	<u>CodCorso</u>	Data	Voto
34321	1	25/02/01	28
34321	2	14/12/01	30
64521	1	12/03/02	24
73321	4	18/01/02	18

Definizione Tabella Corso

```
CREATE TABLE Corso (  
CodCorso    CHAR(6)    PRIMARY KEY,  
Titolo      VARCHAR(30) NOT NULL,  
Docente     VARCHAR(20)  
)
```

<u>CodCorso</u>	Titolo	Docente
1	matematica	Barozzi
2	informatica	Meo
3	ingegneria	Neri
4	Matematica	Bianchi

Vincoli interrelazionali : Integrità referenziale

<u>Matr</u>	Nome	Città	CDip
34321	Luca	Mi	Inf
53524	Giovanni	To	Mat
64521	Emilio	Ge	Ing
73321	Francesca	Vr	Mat

Studente

<u>Matr</u>	<u>CodCorso</u>	Data	Voto
34321	1	25/02/01	28
34321	2	14/12/01	30
64521	1	12/03/02	24
73321	4	18/01/02	18

Esame

<u>CodCorso</u>	Titolo	Docente
1	matematica	Barozzi
2	informatica	Meo
3	ingegneria	Neri
4	Matematica	Bianchi

Corso

Vincoli interrelazionali : Esempio

```
CREATE TABLE Esame (  
  Matr          CHAR(6),  
  CodCorso      CHAR(6),  
  Data          DATE      NOT NULL,  
  Voto          Voto,  
  PRIMARY KEY (Matr, CodCorso)  
)
```


Vincoli interrelazionali : Esempio

```
CREATE TABLE Esame (  
  Matr          CHAR(6),  
  CodCorso      CHAR(6),  
  Data          DATE    NOT NULL,  
  Voto          Voto,  
  PRIMARY KEY (Matr, CodCorso)
```

```
FOREIGN KEY (Matr) REFERENCES Studente
```

```
)
```

Vincoli interrelazionali : Esempio

```
CREATE TABLE Esame (  
  Matr          CHAR(6),  
  CodCorso      CHAR(6),  
  Data          DATE      NOT NULL,  
  Voto          Voto,  
  PRIMARY KEY (Matr, CodCorso)
```

```
FOREIGN KEY (Matr) REFERENCES Studente
```

```
FOREIGN KEY (CodCorso) REFERENCES Corso
```

```
)
```

Vincoli interrelazionali : Esempio

<u>Matr</u>	Nome	Città	CDip
123			
456			
789			
120			

Studente

Una istanza scorretta

Esame

<u>Matr</u>	<u>CodCorso</u>	Data	Voto
123	1	25/02/01	28
123	2	14/12/01	30
123	1	12/03/02	24
120	4	18/01/02	25
456	1	12/03/02	NULL
555	4	18/01/02	23

Viola la chiave

Viola il NULL

Viola integrità referenziale

Vincoli interrelazionali : Esempio

<u>Matr</u>	Nome	Città	CDip
123			
456			
789			
120			

Studente

Una istanza corretta

Esame

<u>Matr</u>	<u>CodCorso</u>	Data	Voto
123	1	25/02/01	28
123	2	14/12/01	30
120	4	18/01/02	25

Vincoli con nomi

A fini diagnostici (e di documentazione) è spesso utile sapere quale vincolo è stato violato a seguito di un'azione sul DB.

A tale scopo è possibile associare dei nomi ai vincoli, ad esempio:

Stipendio **INTEGER CONSTRAINT** StipendioPositivo
CHECK (Stipendio > 0),

CONSTRAINT ForeignKeySedi
FOREIGN KEY (Sede) **REFERENCES** Sedi

Modifiche degli schemi:

Necessarie per garantire l'evoluzione della base di dati a fronte di nuove esigenze.

Ci sono due comandi SQL appositi:

ALTER: modifica oggetti

DROP: cancella oggetti dallo schema

Modifiche degli schemi: ALTER

- **ALTER DOMAIN**, modifica domini
- **ALTER TABLE**, modifica tabelle.

Attenzione: Quando si definisce un nuovo vincolo, deve essere soddisfatto dalla istanza presente, altrimenti viene rifiutato.

Modifiche degli schemi: DROP

DROP DOMAIN, cancellazione del dominio (attributo)

DROP TABLE, cancellazione della tabella

Due opzioni:

- **RESTRICT**, un oggetto (dominio, tabella) non è rimosso se non è vuoto
- **CASCADE**, viene rimosso l'oggetto e tutto ciò che è coinvolto nella definizione dell'oggetto

Modifiche degli schemi : ALTER

Si applica su domini e tabelle:

```
ALTER DOMAIN NomeDominio <
  SET      DEFAULT ValoreDefault |
  DROP     DEFAULT |
  ADD      CONSTRAINT DefVincolo |
  DROP     CONSTRAINT NomeVincolo >
```

Modifiche degli schemi : ALTER

Si applica su domini e tabelle:

```
ALTER TABLE NomeTabella <
  ALTER COLUMN NomeAttributo <
    SET DEFAULT NuovoDefault |
    DROP DEFAULT > |
  DROP COLUMN NomeAttributo |
  ADD COLUMN DefAttributo |
  DROP CONSTRAINT NomeVincolo
  ADD CONSTRAINT DefVincolo >
```

Modifiche degli schemi : DROP

Cancella oggetti DDL, si applica su domini, tabelle, indici, view, asserzioni, procedure,...

DROP < schema, domain, table, view, ...> *NomeElemento*
[**RESTRICT** | **CASCADE**]

Opzioni:

RESTRICT Impedisce drop se gli oggetti comprendono istanze non vuote

CASCADE Applica drop agli oggetti collegati. Potenziale pericolosa reazione a catena

ESERCIZI

Esercizio

Definire un dominio che permetta di rappresentare stringhe di lunghezza massima pari a 256 caratteri, su cui non sono ammessi valori nulli e con valore di default “sconosciuto”.

```
CREATE DOMAIN NomeDominio as DominioElementare  
    [ ValoreDefault ] [ Constraints ]
```

Soluzione:

```
CREATE DOMAIN String as character varying (256)  
default 'sconosciuto' not null
```

Esercizio

Si consideri il seguente schema relazionale:

PRODOTTO(cod-prod, descrizione, prezzo-unitario)

FATTURA(cod-fatt, cliente, cod-prod, quantità)

Implementare in SQL lo schema relazionale dell'esercizio con gli opportuni vincoli di integrità

Esercizio

PRODOTTO(cod-prod, descrizione, prezzo-unitario)

```
CREATE TABLE PRODOTTO (  
  cod-prod char(3) PRIMARY KEY,  
  descrizione char(25),  
  prezzo-unitario decimal(6,2) CHECK (prezzo-unitario > 0)  
)
```

FATTURA(cod-fatt, cliente, cod-prod, quantità)

```
CREATE TABLE FATTURA (  
  cod-fatt char(3), PRIMARY KEY  
  cliente char(25),  
  cod-prod char(3) REFERENCES Prodotto (cod-prod),  
  quantità smallint CHECK (quantità > 0),  
)
```

Esercizio

Dare le definizioni SQL delle tre tabelle

FONDISTA(Nome, Nazione, Età)

GAREGGIA(NomeFondista, NomeGara, Piazzamento,
DataGara)

GARA(Nome, Data, Luogo, Nazione, Lunghezza)

rappresentando in particolare i vincoli di foreign key della tabella GAREGGIA.

Riassunto

1. Linguaggio SQL
2. Istruzioni (o comando) del linguaggio
3. L'istruzione CREATE Linguaggio per la definizione di tabelle e domini
4. Vincoli intrarelazionali
5. Vincoli interrelazionali
6. Politica di reazione alle violazioni
7. L'istruzione ALTER
8. L'istruzione DROP